WebAuthn & PassKeys

Passwordless (And more secure) Logins

What is WebAuthn?

- **Standardized Protocol:** Web Authentication (WebAuthn) is a W3C-published web standard.
- **API Definition:** Provides an API for websites to authenticate using passkeys and establishes guidelines for authenticator functionality.
- **Identity Verification:** Utilizes digital signatures to verify user identity securely.
- **Cryptographic Foundation:** Replaces traditional passwords and SMS codes with asymmetric (public-key) cryptography.
- **Use Cases:** Supports user registration, identity verification, and multi-factor authentication across web platforms.
- **Key Storage:** Relies on authenticators to securely store cryptographic keys.

What is PassKey?

- Non-technical term for WebAuthn credentials
- A public/private key pair associated with a specific user account on a specific website.
- (RP ID = example.com, User Handle = 12345, Username = alice@example.com, Credential ID, PrivateKey)

What is WebAuthn Authenticator?

It is responsible for generating and storing the WebAuthn credential (Passkey). Types of Authenticators:

- Platform Authenticators
  - Built into the Operating System of the device
  - Make use of Hardware Security Features - Trusted Execution Environment, Trusted Platform Module
  - Often sync credentials b/w devices for ease of use
  - Examples - Android, Apple Keychain, Windows Hello
- Roaming Authenticators
  - Separate Hardware Device authenticates the user
  - Connects over USB, Bluetooth Low Energy or Near Field Communications (NFC)
  - Examples - Most smartphone, Dedicated Physical Security Keys
- Password Managers can also be used as an authenticator, often with cloud sync

Why WebAuthn?

- Secure credential generation and storage: WebAuthn generates unique credentials for each website using robust algorithms, storing them securely in trusted authenticators. This eliminates common vulnerabilities such as:
  - Weak passwords that can be easily brute-forced due to insufficient length.
  - Predictable passwords vulnerable to dictionary attacks (e.g., "password", "12345678").
  - Guessable passwords based on personal information (e.g., birthdates, addresses).
  - Poor client-side password storage (e.g., written down, stored in phone contacts).
  - Password reuse across multiple websites, as WebAuthn credentials are specific to individual websites by design.
  - Inadequate server-mandated password requirements (e.g., overly lax or restrictive criteria, arbitrary maximum length limits, limited charsets).
  - Restrictions preventing password manager auto-fill features.

Why WebAuthn?

- No server-side credential storage: The private part of a credential is never stored on a server, eliminating risks and vulnerabilities such as:
  - Insecure password storage in databases (e.g., plaintext or relying on weak hash-based algorithms/constructions).
  - Database leaks exposing passwords.
  - Mandatory, ineffective periodic password changes.
- Unique credentials for each website: WebAuthn ensures credentials are unique per website, eliminating the following risks and vulnerabilities:
  - Credential stuffing attacks, where attackers use credentials from one data breach across multiple sites.
  - Phishing attacks, as credentials cannot be reused or misapplied to different websites.

Components of WebAuthn

- **Relying Party (RP)**
  - The website which wants to authenticate (verify the identity of) the user
- **WebAuthn Client**
  - The browser from where the user is trying to log into the website.
- **WebAuthn Authenticator**
  - the authenticator which facilitates the secure generation and usage of WebAuthn Credentials (PassKeys).

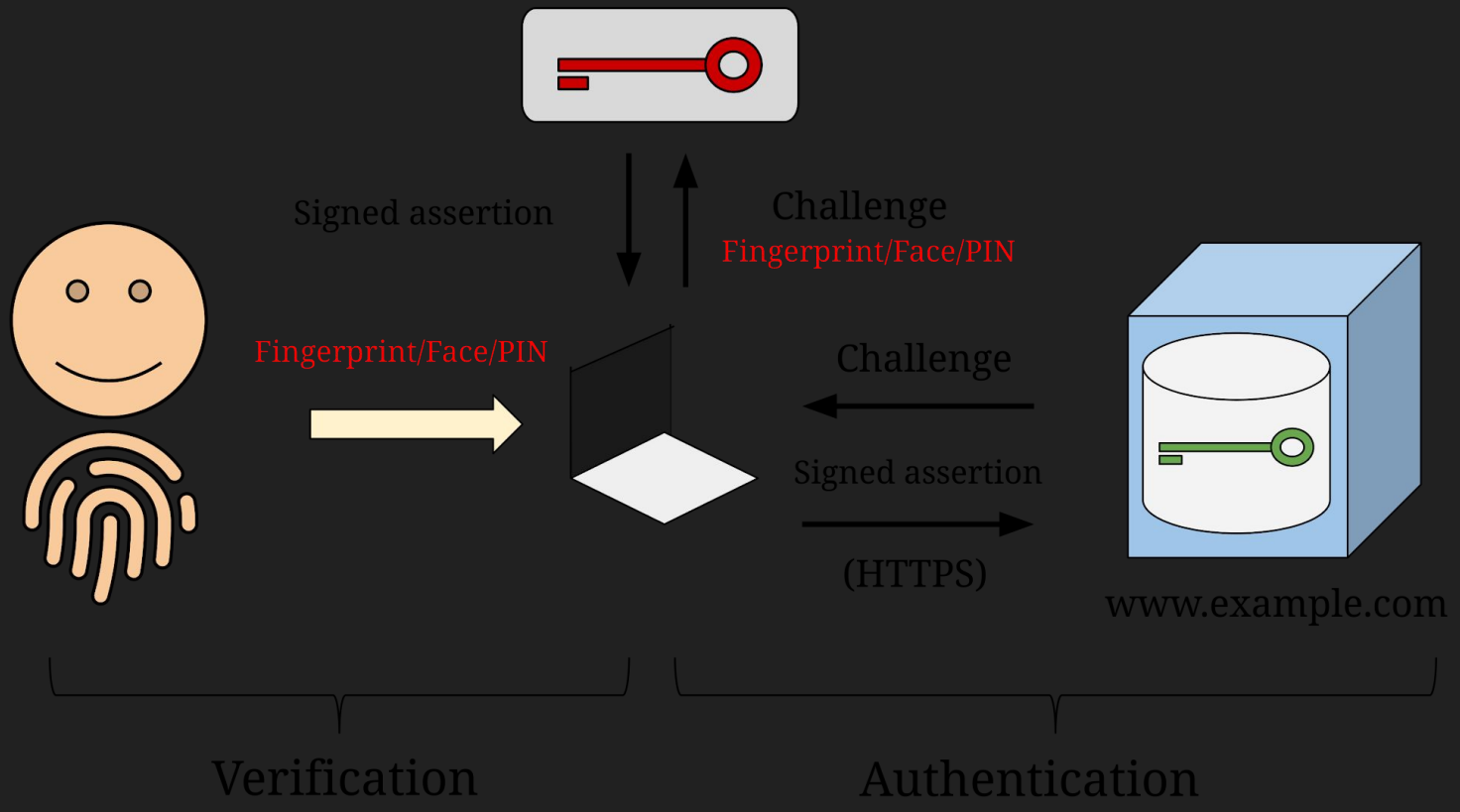
WebAuthn Flows

Registration Flow

The user registers a credential to a relying party. This can be the existing user or a new user, it depends on how the relying party is using WebAuthn. Nowadays, most of the websites want their existing users to register their passkeys (credentials) with them, so that this later can be used for authentication.

Authentication Flow

After a user has registered with WebAuthn, they can authenticate (login) with the service.



Registration Flow

1. Client sends Registration Request to RP Server
2. RP Server replies back with Registration Options
    a. a random **challenge** (prevents replay attacks)
    b. the **Relying Party (RP) ID** (usually the domain)
    c. a **User ID** (an opaque identifier)
    d. supported cryptographic algorithms
    e. authenticator preferences (resident key, user verification, etc.)

3. Client invokes the **WebAuthn API**

```
let credential = await navigator.credentials.create({
  publicKey: {
    challenge: new Uint8Array([117, 61, 252, 231, 191, 241 /* … */]),
    rp: { id: "acme.com", name: "ACME Corporation" },
    user: {
      id: new Uint8Array([79, 252, 83, 72, 214, 7, 89, 26]),
      name: "jamiedoe",
      displayName: "Jamie Doe",
    },
    pubKeyCredParams: [{ type: "public-key", alg: -7 }],
  },
});
```

4. Authenticator Verifies the user

- Fingerprint, Face ID, PIN, Key Touch, etc.

5. Authenticator Generates a Key Pair and returns an attestation (Next page for the response from authenticator) - Type is `PublicKeyCredential`

6. The Client (browser) sends the attestation to the RP Server

```
{
  "id": "", // Base64Url Encoding of rawId
  "rawId": [], // An ArrayBuffer that holds the globally unique identifier for this PublicKeyCredential. This identifier can be used to look up credentials for future calls to navigator.credentials.get().
  "type": "public-key", // Always for PublicKeyCredential
  "authenticatorAttachment": "cross-platform", // mechanism by which the WebAuthn implementation is attached to the authenticator ("platform" or "cross-platform")
  "response": { // Type - AuthenticatorAttestationResponse
    "attestationObject": [], // ArrayBuffer containing authenticator data and an attestation statement for a new key pair generated by the authenticator
    "clientDataJSON": “{}” // JSON-Compatible serialization of the data passed from the browser to the authenticator in order to generate this credential
  }
}
```

7. Server verifies the information:

- the challenge matches
- the origin is correct
- the RP ID hash matches
- the signature is valid
- user presence/user verification requirements
- attestation (if required)

8. Server stores the following information:

- User Id
- Credential Id
- Public Key
- Signature Counter

Authentication Flow

1. Client sends Login Request to the RP Server
2. RP Server replies back with login options:
    a. Challenge
    b. RP ID
    c. List of allowed credentials (if username is already known)

3. Client asks the authenticator to sign the challenge

```
let credential = await navigator.credentials.get({
  publicKey: {
    challenge: new Uint8Array([139, 66, 181, 87, 7, 203 /* …
*/]),
    rpId: "acme.com",
    allowCredentials: [
      {
        type: "public-key",
        id: new Uint8Array([64, 66, 25, 78, 168, 226, 174 /*
… */]),
      },
    ],
    userVerification: "required",
  },
});
```

4. If the authenticator contains one of the given credentials (if allowCredentials is not empty) for the given rpId:

4.1. Asks the user to select one if multiple matches are found

4.2. Sign the challenge

4.3. Ask the user for consent

5. Returns PublicKeyCredential (see next page)

6. The client sends the credential to the RP server

7. The server verifies the signature and other data

```
{

  "id": "", // Base64Url Encoding of rawId

  "rawId": [], // An ArrayBuffer that holds the globally unique identifier for this PublicKeyCredential. This identifier can be used to look up credentials for future calls to navigator.credentials.get().

  "type": "public-key", // Always for PublicKeyCredential

  "authenticatorAttachment": "cross-platform", // mechanism by which the WebAuthn implementation is attached to the authenticator ("platform" or "cross-platform")

  "response": { // Type - AuthenticatorAttestationResponse

    "authenticatorData": [], // An ArrayBuffer containing information from the authenticator such as the Relying Party ID Hash (rpIdHash), a signature counter, test of user presence and user verification flags, and any extensions processed by the authenticator.

    "clientDataJSON": “{}”,

    "signature": [], // An assertion signature over authenticatorData and clientDataJSON. The assertion signature is created with the private key of the key pair that was created during the registration flow and can be verified using the public key of that same key pair.

    "userDataHandle": [], // An ArrayBuffer containing an opaque user identifier, specified as user.id in the options passed to the originating registration flow call.

  }

}
```

Type of Credentials

1. **Discoverable Credentials** (Resident Keys)
2. **Non-discoverable Credentials** (Non-resident Keys)

```
navigator.credentials.create({

  // … other things

  authenticatorSelection: {

    requireResidentKey: true, // or false

    residentKey: “discouraged”, // or “preferred”, “required”

}
```

Non-discoverable Credentials

- Stored at the RP Server:
  - Username
  - ID of RP
  - Encrypted Private Key
  - Public Key
- Stored in the Authenticator:
  - Master Key

Flows for Server-side credentials

1. **Registration** steps at authenticator:
    a. Generates the key pair
    b. Encrypts the private key and other data with the master key
    c. Return the encrypted private key along with other data to the RP
2. **Authentication** steps:
    a. The RP passes the credential ID into the `CredentialsContainer.get()` call
    b. The authenticator encrypts the credential ID value into the private key and other data, using the authenticator's stored **master key**.
    c. The authenticator uses the private key to sign an assertion.

Trade-offs of Non-discoverable Credentials

Advantages

- Authenticator doesn’t have to store any credential-specific data
- It could support an essentially infinite number of credentials

Disadvantage

- The user must first supply the username they want to sign in as
- Can not be discovered automatically

Discoverable Credentials

The authenticator itself stores:

- The private key material used to generate assertions.
- The username associated with the credential.
- The ID of the RP associated with the credential.

Advantages

- The browser can discover the credentials associated with the RP
- Display associated usernames to the user
- Invite the user to choose the one they want to sign in with

 Passkeys are always Discoverable Credentials.

References

- WebAuthn - Wikipedia
- Web Authentication: An API for accessing Public Key Credentials - Level 2
- Web Authentication API - Web APIs | MDN
- About passkeys - Github Docs
- A Tour of WebAuthn - Adam Langlay
- PublicKeyCredentialCreationOptions - Web API | MDN


Spring Security IN ACTION
JAVA FUELS ME.
FIDO2/WebAuthn
Spring Security
FIDO2
WebAuthn
Spring Security

Thank You